While the Rails framework and third-party testing tools allow you to interact with things like mailers, controller actions, and view templates in your test environment, the issues of isolation and size still exist. A discussion of how to best test these parts of the Rails code often turns into one of the following two discussions:

- What should be tested in integration tests, and what should be tested in unit tests?
- What code belongs in the Rails-specific controller, view, mailer, job, or whatever, and what should be extracted into a different object?

The answers to these questions tend to be related—the more you want to keep code in the Rails constructs, the more likely you are to also want to use integration tests rather than unit tests.

For Rails 5, the Rails core team made a strong statement of preference by deprecating some commonly used features of controller tests in favor of integration or system tests. This is consistent with Rails founder David Heinemeier Hansson's views that extra layers within Rails are unnecessary.

Testing Routes

Let's start the tour with a part of the Rails layer that's not commonly tested in isolation, but where testing can sometimes be valuable. Although the basics of Rails routing are simple, the desire to customize Rails's response to URLs can lead to confusion about exactly what your application is going to do when converting between a URL and a Rails action. Rails provides a way to specify route behavior in a test.

Routing tests are not typically part of my TDD process—usually my integration test implicitly covers the routing. That said, sometimes routing gets complicated and has some logic of its own (especially if you're trying to replicate an existing URL scheme), so it's nice to have this as part of your test suite.

RSpec-Rails puts routing tests in the spec/routing directory. The primary matcher that RSpec-Rails uses for route testing is route_to. Here's a sample test that includes all seven default RESTful routes for the project resource:

display/01/spec/routing/project_routing_spec.rb

```
require "rails_helper"

RSpec.describe "project routing", :aggregate_failures, type: :routing do

  it "routes projects" do
    expect(get: "/projects").to route_to(
      controller: "projects", action: "index")
```